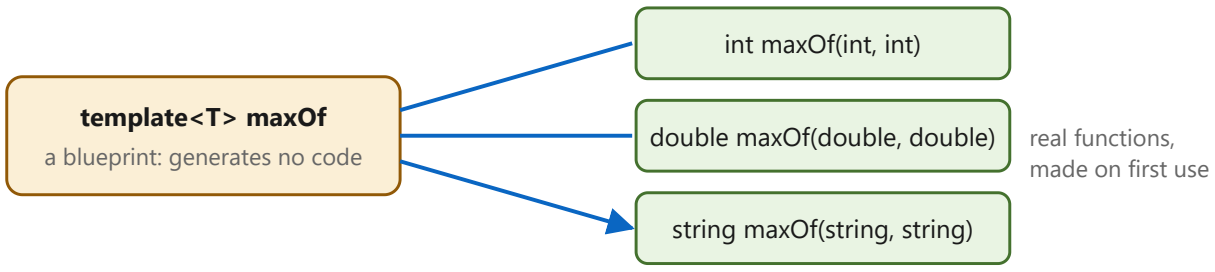
C++ Templates

Write code once, let the compiler generate it per type. From a first function template to forwarding references, variadics and type traits: the topics the job description lists.

Why templates, and function templates



Compile-time polymorphism: no runtime cost, more code per type.

Without templates: `maxInt`, `maxDouble`, `maxString`... A **template** takes the type as a parameter:

```
template <typename T>
T maxOf(T a, T b) {
    return a < b ? b : a;
}

maxOf(3, 7);          // compiler deduces T = int
maxOf(2.5, 1.5);      // T = double
maxOf<double>(3, 2.5);// explicit: T = double, the int is converted
```

The template is not compiled itself. Each new type **instantiates** a real function: no runtime cost, more binary size, and errors appear only when a bad type is used.

Pick a type:

TRY IT **Instantiate maxOf**

```
template <typename T>
T maxOf(T a, T b) { return a < b ? b : a; }
```

Call: maxOf(3, 7)

Deduced: T = int
Generated: `int maxOf(int a, int b)`. Returns 7.

KEY POINT Templates: **compile-time** (static) polymorphism, fastest, inlinable, more code, types known at compile time. Virtual: **runtime**, one code path, mixed objects, a vtable indirection.

READ MORE

C++ Primer (Lippman, Lajoie, Moo)

- 16.1 Defining a template (PDF pages 681 to 706)

Template argument deduction

When you call a function template, the compiler works out `T` from the arguments. The rule depends on the **shape of the parameter**, and it is the same rule `auto` uses.

PARAMETER	CALL	DEDUCED `T`	RULE
`T x` (by value)	`f(ci)` with `const int ci`	`int`	a copy: top-level `const` and references are **dropped**
`T x` (by value)	`f(arr)` with `int arr[3]`	`int*`	arrays and functions **decay** to pointers

`T& x`	`f(ci)`	`const int`	the reference keeps the argument's `const`
`T& x`	`f(arr)`	`int[3]`	an array keeps its size (usable to get its length)
`const T& x`	`f(5)`	`int`	binds temporaries; `const` is not part of `T`
`T&& x`	`f(i)` / `f(5)`	`int&` / `int`	forwarding reference (see Forwarding references)

Compiled and checked with `static_assert`: by value, `const int` deduced `int` and the array deduced `int*`; by reference they deduced `const int&` and `int(&)[3]`.

- Deduction looks only at the **arguments**, never at the return type, and **no conversions** happen: `maxOf(3, 2.5)` fails because `T` would be both `int` and `double`.
- Default arguments and braced lists do not take part in deduction: `f({1, 2})` cannot deduce `T`. Name `T` explicitly: `f<std::vector<int>>({1, 2})`.
- Name the type yourself when needed: `maxOf<double>(3, 2.5)`.

Class template argument deduction (C++17)

```
std::pair p{1, 2.5};                    // std::pair<int, double>
std::vector v{1, 2, 3};                 // std::vector<int>
std::array a{1, 2, 3};                  // std::array<int, 3>

template <typename T> struct Box { T value; };
template <typename T> Box(T) -> Box<T>; // a user-defined deduction guide
(needed for aggregates before C++20)
Box b{std::string("x")};                // Box<std::string>
```

The compiler deduces the class's arguments from the constructor call. A **deduction guide** tells it how when the constructors alone are not enough. All of the above was compiled and checked.

> **WHEN DEDUCTION FAILS** The error says "couldn't deduce template parameter". Look for conflicting arguments, a braced list, or a parameter that does not mention `T` at all (e.g. only the return type does).

READ MORE

C++ Primer (Lippman, Lajoie, Moo)

- 16.2 Template argument deduction (PDF pages 707 to 722)
- 16.3 Overloading and templates (PDF pages 723 to 727)

Effective Modern C++ (Meyers)

- Item 1, Understand template type deduction (PDF pages 27 to 36)

Class templates

Same idea for classes: `vector<T>`, `map<K,V>`, `unique_ptr<T>`.

```
template <typename T, std::size_t Capacity = 8>      // a type parameter, a
NON-TYPE parameter, with a default
class Stack {
    std::array<T, Capacity> items_;
    std::size_t size_ = 0;
public:
    bool push(const T& v) {
        if (size_ == Capacity) return false;
        items_[size_++] = v;
        return true;
    }
    T pop() { return items_[--size_]; }
    bool empty() const { return size_ == 0; }
};

Stack<int> a;               // Stack<int, 8>
Stack<std::string, 100> b;  // a different, unrelated type

std::pair p{1, 2.5};        // C++17 CTAD: class template argument deduction
→ std::pair<int, double>
std::vector v{1, 2, 3};     // std::vector<int>
```

- **Non-type parameters** are compile-time constants (how `std::array<int,5>` knows its size).
- Each instantiation is a separate, unrelated class. Members are instantiated only if called.

WHY TEMPLATES LIVE IN HEADERS The compiler needs the body at instantiation. In a `.cpp`, others see only a declaration: linker errors. Use headers, or explicit instantiation (`template class Stack<int>;`).

READ MORE

C++ Primer (Lippman, Lajoie, Moo)

- 16.1 Defining a template (PDF pages 681 to 706)

Programming: Principles and Practice (Stroustrup)

- 18.1 to 18.3 Templates, generalizing Vector, range checking and exceptions (PDF pages 527 to 541)

Specialization

A **specialization** replaces the generic template for specific arguments.

```
template <typename T>
struct Printer {                              // primary template
    static void print(const T& v) { std::cout << v << '\n'; }
};

template <>                                   // full specialization for bool
struct Printer<bool> {
    static void print(bool v) { std::cout << (v ? "yes" : "no") << '\n'; }
};

template <typename T>
struct Printer<T*> {                          // PARTIAL specialization: any
pointer type
    static void print(T* p) { std::cout << "ptr to " << *p << '\n'; }
};
```

KIND	MEANING	ALLOWED FOR
Full specialization	all parameters fixed: `template<> ... Printer<bool>`	class and function templates

Partial specialization	a pattern: `Printer<T*>`	**class templates only**

FUNCTIONS: PREFER OVERLOADS No partial specialization for function templates. Overload instead; an exact non-template match wins. Function specializations do not take part in overload resolution.

READ MORE

C++ Primer (Lippman, Lajoie, Moo)

- 16.5 Template specializations (PDF pages 735 to 743)

Variadic templates and fold expressions

A **parameter pack** (`typename ... Args`) takes any number of arguments; `args ...` expands it. Before C++17: recursion with a base case.

```
template <typename T>
T sum(T v) { return v; }                                    // base case: one
argument

template <typename T, typename ... Rest>
T sum(T first, Rest ... rest) { return first + sum(rest ...); }    // peel one
off, recurse

sum(1, 2, 3);
```

The compiler instantiates a chain of real functions:

STEP THROUGH **Recursive instantiation of sum(1, 2, 3)**

```
1  template <typename T>
2  T sum(T v) { return v; }
3  template <typename T, typename... Rest>
4  T sum(T first, Rest... rest) { return first + sum(rest...); }
5
6  sum(1, 2, 3);
```

STEP	RUNS	WHAT HAPPENS	MEMORY AFTER (CHANGED IN BOLD)
1	line 6 `sum(1, 2, 3);`	We call `sum(1, 2, 3)`. The compiler deduces `T = int` and `Rest = {int, int}`.	
2	line 4 `T sum(T first, Rest... rest) { return first + sum(rest...); }`	It **instantiates** the variadic overload for these types. Its body contains a call `sum(rest...)` = `sum(2, 3)`, which needs another instantiation.	**`sum(1, 2, 3) = 1 + sum(2, 3)`**
3	line 4 `T sum(T first, Rest... rest) { return first + sum(rest...); }`	A second instantiation, for two arguments. It calls `sum(3)`.	`sum(1, 2, 3) = 1 + sum(2, 3)` | **`sum(2, 3) = 2 + sum(3)`**
4	line 2 `T sum(T v) { return v; }`	With one argument the **base case** overload is chosen (it is the better match), and the recursion stops. Three real functions now exist, all generated at compile time.	`sum(1, 2, 3) = 1 + sum(2, 3)` | `sum(2, 3) = 2 + sum(3)` | **`sum(3) = return 3`**
5	line 4 `T sum(T first, Rest... rest) { return first + sum(rest...); }`	At runtime the calls unwind: `sum(2, 3)` returns 2 + 3 = 5.	**`sum(1, 2, 3) = 1 + 5`** | **`sum(2, 3) = 2 + 3 = 5`** | `sum(3) = return 3`
6	line 4 `T sum(T first, Rest... rest) { return first + sum(rest...); }`	The result is 6. Optimising compilers usually inline the whole chain into a single constant. With a C++17 fold expression, `(args + ...)`, you get the same result without writing the recursion. prints: `6\n`	**`sum(1, 2, 3) = 1 + 5 = 6`** | `sum(2, 3) = 2 + 3 = 5` | `sum(3) = return 3`

C++17 **fold expressions** replace the recursion:

```
template <typename ... Args>
auto sum(Args ... args) { return (args + ... ); }          // unary right fold:
a + (b + (c ...))

template <typename ... Args>
void print(Args&& ... args) { ((std::cout << args << ' '), ... ); }   // comma
fold: do it for each arg

sizeof ... (Args);                                          // number of
elements in the pack
```

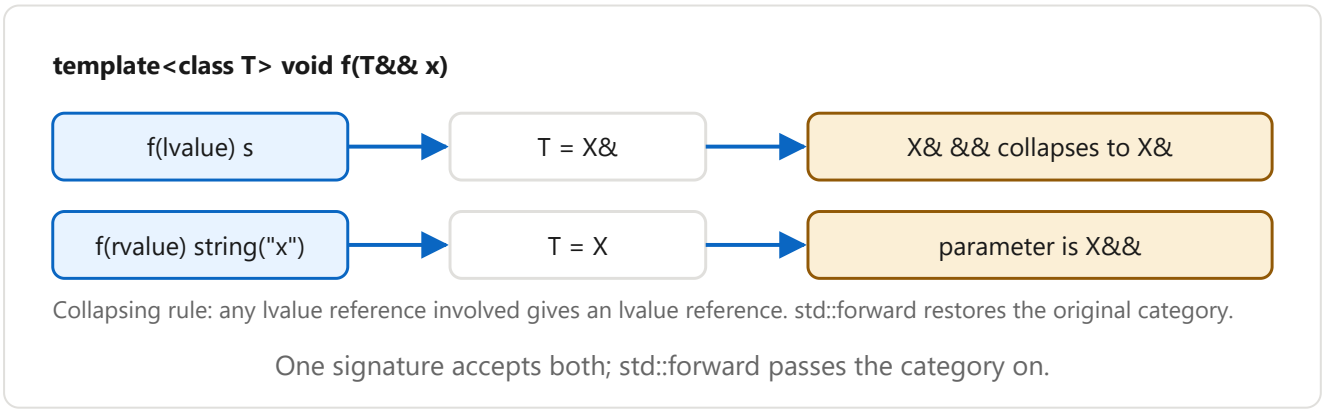
Behind `make_unique`, `make_shared`, `emplace_back`, `tuple`, `thread`, `function`.

READ MORE

C++ Primer (Lippman, Lajoie, Moo)

- 16.4 Variadic templates (PDF pages 728 to 734)

Forwarding references and perfect forwarding



`T&&` **in a function template** is a **forwarding reference**, not an rvalue reference: it binds to anything and remembers lvalue vs rvalue. Explore:

EXPLORER **What does the compiler deduce for** `template<class T> void f(PARAM)` **?**

Parameter PARAM: T Argument: int x; f(x) // lvalue int

Deduced `T = int`, so the parameter type is `int`.

Is it really a forwarding reference?

Only when `T` is deduced for *that* function and the form is exactly `T&&` or `auto&&`:

DECLARATION	FORWARDING REFERENCE?
`template<class T> void f(T&& x)`	yes: `T` is deduced for `f`
`auto&& x = expr;`	yes
`template<class T> void f(std::vector<T>&& v)`	no: not exactly `T&&`, so it binds rvalues only
`template<class T> void f(const T&& x)`	no: `const` disqualifies it
`template<class T> struct S { void f(T&& x); };`	no: `T` is fixed by the class, so nothing is deduced for `f`
`void f(Widget&& w)`	no: a plain rvalue reference

Reference collapsing

Reference to reference collapses: **any lvalue reference makes it an lvalue reference; only `&& + &&` stays rvalue.**

WRITTEN	COLLAPSES TO
`T& &`, `T& &&`, `T&& &`	`T&`
`T&& &&`	`T&&`

Perfect forwarding

A wrapper's parameter has a *name*, so it is an lvalue and the callee would always copy. `std::forward` restores the category:

```
template <typename T, typename... Args>
std::unique_ptr<T> makeUnique(Args&&... args) {                  //
forwarding references
    return std::unique_ptr<T>(new T(std::forward<Args>(args)...));  //
lvalues stay lvalues, rvalues stay rvalues
}

std::string s = "abc";
auto a = makeUnique<Widget>(s);              // s is an lvalue  → Widget
copies it
auto b = makeUnique<Widget>(std::string("x"));// temporary rvalue → Widget
MOVES it
```

> **STD::MOVE VS STD::FORWARD** `move` *always* casts to rvalue. `forward<T>` casts to rvalue *only if* the argument was one. Use `forward` only on a forwarding reference, once.

Step through perfect forwarding:

STEP THROUGH **Perfect forwarding: copy for lvalues, move for rvalues**

```
1  template <typename T>
2  void wrap(T&& arg) { target(std::forward<T>(arg)); }
3
4  std::string s = "abc";
5  wrap(s);
6  wrap(std::string("x"));
```

STEP	RUNS	WHAT HAPPENS	MEMORY AFTER (CHANGED IN BOLD)
1	line 5 `wrap(s);`	An ordinary named string: an **lvalue**.	**`s = "abc"`**
2	line 6 `wrap(std::string("x"));`	`wrap(s)`: s is an lvalue, so **T = string&**. The parameter `T&&` collapses to `string&` and binds to s without copying.	`s = "abc"` | **`T = string&`** | **`arg = alias`**
3	line 2 `void wrap(T&& arg) { target(std::forward<T>(arg)); }`	`std::forward<string&>(arg)` keeps it an **lvalue**, so target picks the copying overload. Inside wrap, arg has a name and is always an lvalue; forward restores what the caller passed. prints: `target(const string&): COPY\n`	`s = "abc"` | `T = string&` | `arg = alias`
4	line 7	`wrap(std::string("x"))`: a temporary is an **rvalue**, so **T = string** and the parameter is a true `string&&` bound to the temporary.	`s = "abc"` | **`temp = "x"`** | **`T = string`** | `arg = alias`
5	line 2 `void wrap(T&& arg) { target(std::forward<T>(arg)); }`	`std::forward<string>(arg)` casts to an **rvalue**, so target picks the move overload. Same wrapper, correct behaviour for both: that is perfect forwarding. (`std::move` would have moved in *both* cases.) prints: `target(string&&): MOVE\n`	`s = "abc"` | **`temp = moved-from`** | `T = string` | `arg = alias`

READ MORE

C++ Primer (Lippman, Lajoie, Moo)

- 16.2 Template argument deduction (PDF pages 707 to 722)

Effective Modern C++ (Meyers)

- Item 23, Understand std::move and std::forward (PDF pages 176 to 182)
- Item 24, Distinguish universal references from rvalue references (PDF pages 182 to 186)
- Items 26 to 27, and 30 Overloading on universal references; perfect forwarding failure cases (PDF pages 195 to 215 and 225 to 232)

C++ Move Semantics (Josuttis)

- 9.1 to 9.3 Perfect forwarding; universal versus rvalue references (PDF pages 159 to 166)
- 9.4 to 9.5 Overload resolution with universal references; perfect forwarding in lambdas (PDF pages 167 to 170)
- Chapter 11 Perfect passing with auto&& (PDF pages 191 to 202)

Alias templates, variable templates and more

TOOL	EXAMPLE	USE
Alias template	`template <typename T> using Table = std::map<std::string, std::vector<T>>;`	a shorter name that still takes a parameter; unlike `typedef` it can be templated
Variable template (C++14)	`template <typename T> constexpr T zero = T{};`	a constant per type: `zero<int>`, `zero<double>`. The standard's `std::is_integral_v<T>` is one
Template template parameter	`template <template <typename ...> class C, typename T>`	pass a *template* (`std::vector`) rather than a type: `makeTwo<std::vector>(3, 4)`

Non-type parameter	`template <typename T, std::size_t N>`	a value as parameter; `auto` accepts any type of value in C++17: `template <auto V>`
Abbreviated template (C++20)	`void f(auto x)`	each `auto` parameter is an implicit template parameter

```
template <typename T> using Table = std::map<std::string, std::vector<T>>;
Table<int> t;                                   // std::map<std::string,
std::vector<int>>

template <typename T> constexpr T zero = T{};        // zero<int> is 0,
zero<double> is 0.0

template <template <typename ...> class Container, typename T>
Container<T> makeTwo(T a, T b) { Container<T> c; c.push_back(a);
c.push_back(b); return c; }
auto v = makeTwo<std::vector>(3, 4);                 // a vector<int> with
two elements
```

Unpacking with `std::index_sequence`

To call a function with the elements of a `std::tuple` you need the indices `0, 1, 2 ...` as a pack. `std::make_index_sequence<N>` generates it:

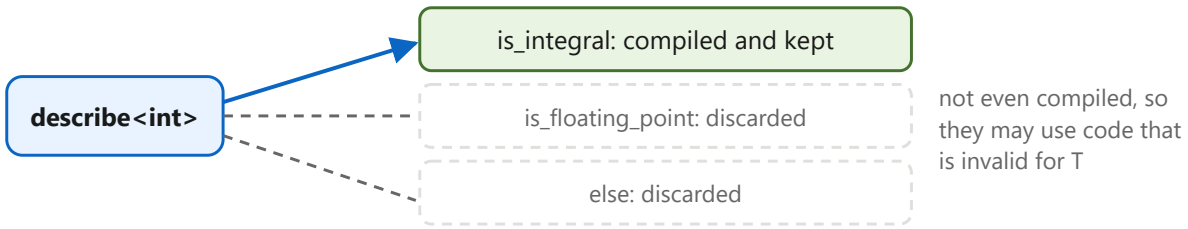
```
template <typename F, typename Tuple, std::size_t ... I>
auto applyImpl(F f, const Tuple& t, std::index_sequence<I ...>) { return
f(std::get<I>(t) ...); }   // expands to get<0>, get<1>, ...

template <typename F, typename Tuple>
auto applyTuple(F f, const Tuple& t) { return applyImpl(f, t,
std::make_index_sequence<std::tuple_size_v<Tuple>>{}); }

applyTuple([](int x, int y, int z) { return x + y + z; }, std::tuple{1, 2,
3});   // 6 (compiled)
```

C++17 ships this as `std::apply`. Know the idea: a pack of integers lets you expand an expression once per element.

Type traits and `if constexpr`



A normal if would fail to compile as soon as one branch is invalid for T.

A template often needs to ask about its type. `<type_traits>` answers at compile time (`std::is_integral_v<T>`), and `if constexpr` (C++17) keeps only the matching branch:

```
template <typename T>
std::string describe(const T& v) {
    if constexpr (std::is_integral_v<T>)            return "integer "   +
std::to_string(v);
    else if constexpr (std::is_floating_point_v<T>) return "float "     +
std::to_string(v);
    else                                            return "something else";
}
```

Traits, SFINAE and `enable_if`, concepts, `constexpr`, `consteval` and template metaprogramming have their own lesson: Type Traits & Compile-time C++.

READ MORE

Effective Modern C++ (Meyers)

- Items 26 to 27, and 30 Overloading on universal references; perfect forwarding failure cases (PDF pages 195 to 215 and 225 to 232)

C++ Move Semantics (Josuttis)

- Chapter 12 Perfect returning with decltype(auto) (PDF pages 203 to 210)

Pitfalls and good practice

PITFALL	EXPLANATION / FIX
Huge unreadable error messages	The error is reported deep inside an instantiation. Read the *first* error and the line "required from here". Concepts or `static_assert` on traits give early, clear errors.
Template body in a `.cpp` file	Linker errors. Put the definition in the header.
`typename` for dependent types	Inside a template, `T::iterator` could be a type or a value. Write `typename T::iterator it;`.
Members of a dependent base class not found	Write `this->member` or `Base<T>::member`.
Code bloat	Every distinct `T` generates more machine code. Move type-independent logic into a non-template helper.
Deduction conflicts: `maxOf(3, 2.5)`	Two different deduced `T`s fail. Cast, specify `maxOf<double>`, or use two parameters.
`maxOf("a", "b")` with `const char*`	Compares pointer addresses, not text. Use `std::string` or `std::string_view`.

Notes: Module 4; modern-cpp-features.

Cheat sheet

▼ **How are template arguments deduced?**

From the arguments of the call, by the shape of the parameter. By value (`T x`): `const` and references are dropped and arrays decay to pointers. By reference (`T& x`): `const` and the array size are kept. `T&&` is a forwarding reference. No conversions take part, so `maxOf(3, 2.5)` fails; name `T` explicitly.

▼ What are alias templates and variable templates?

An alias template is a parameterised `using` (`template<class T> using Vec = std::vector<T>;`), which a `typedef` cannot be. A variable template is a constant or variable per type (`template<class T> constexpr T zero = T{};`); `std::is_integral_v` is one.

▼ What happens when you use a function template?

The compiler deduces the template arguments from the call, then **instantiates** a concrete function for that combination of types (once per distinct combination across the program), and compiles it like normal code. The template itself generates no code.

▼ What is a forwarding reference?

A `T&&` parameter where `T` is deduced for that very function template (or `auto&&`). It binds to lvalues and rvalues alike: for an lvalue `T` becomes `X&` and reference collapsing makes the parameter `X&`; for an rvalue `T` is `X` and the parameter is `X&&`. `std::forward` then preserves that category.

▼ What is perfect forwarding, and why do we need it?

Passing arguments through a wrapper to another function without changing their value category or constness, so the callee can still choose copy vs move correctly. Without it you would need an overload per combination of lvalue/rvalue/const for every parameter.

▼ Explain SFINAE.

If substituting deduced types into a function template's signature yields an invalid type or expression, that candidate is removed from the overload set rather than raising an error. `std::enable_if` exploits it to enable a function only for suitable types. C++17 `if constexpr` and C++20 concepts are the clearer modern tools.

▼ Full vs partial specialization?

Full: all template arguments fixed. Partial: a pattern with some parameters still free (`Foo<T*>`), allowed for class templates and variable templates but not for function templates (use overloading).

▼ Why must template definitions be in headers?

The compiler needs the body to instantiate it in each translation unit that uses a particular type. If only the declaration is visible, no instantiation happens and the linker cannot find the function. Alternative: explicit instantiation in one `.cpp`.

▼ What is a variadic template and how do you process the pack?

A template taking zero or more arguments via a parameter pack (`typename... Args`). Expand with `args...`. Process by recursion with a base case, or in C++17 with a fold expression like `(args + ...)`.

▼ Templates vs inheritance for polymorphism?

Templates give static (compile-time) polymorphism: no runtime cost and inlinable, but the type must be known when compiling and each type duplicates code. Inheritance with virtual functions gives dynamic (runtime) polymorphism: one code path and heterogeneous collections, at the cost of an indirection and heap-allocated objects.